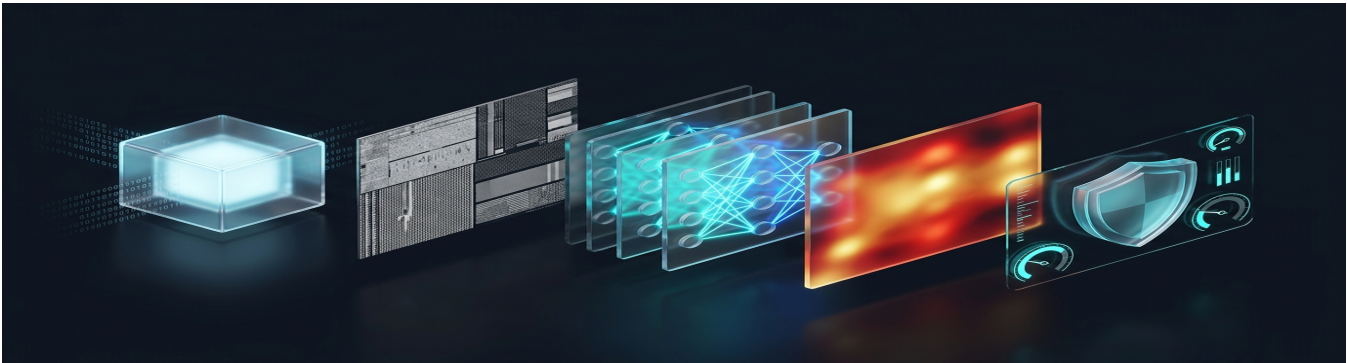


MALVISION: IMAGE-BASED MALWARE CLASSIFICATION

Complete Technical Documentation, Mathematical Formulations & Architecture Blueprint

Model Architecture: ResNet-18 (Grayscale Adapted)	Test Accuracy: 97.86% (1,401 test samples)	Classes: 26 Malware Families
Dataset: Maling Benchmark (9,339 images)	Explainability: Layer4 Grad-CAM Heatmaps	Interface: Streamlit Web Threat Hunter



1. Executive Summary & Core Mission

MalVision is a deep-learning cybersecurity threat hunting platform that identifies and classifies malware without executing or disassembling suspicious software. By transforming raw executable files (.exe, .dll, .bin) into 2D grayscale byteplot textures, MalVision formulates cyber threat detection as an advanced computer vision task, achieving **97.86% test accuracy across 26 real-world malware families** with sub-50-millisecond inference time.

The Airport X-Ray Scanner Analogy: Airport security screeners do not unpack every passenger's bag by hand, nor do they wait for an item to detonate to know if it is dangerous. Instead, luggage passes through an X-ray scanner that produces a density image where specialists recognize prohibited shapes. **MalVision applies this exact principle to computer files:** converting raw executable code into visual textures where a ResNet-18 neural network instantly spots the unique structural blueprints of malware families.

2. Problem Statement: Why Existing Antivirus Fails

Current endpoint security relies heavily on static hash signatures (MD5/SHA256) and dynamic sandbox execution. Signatures fail because polymorphic crypters alter byte values daily, producing new hashes with zero code changes. Sandboxing takes 3-10 minutes per file, requires expensive infrastructure, and is defeated by malware sleep and anti-VM routines.

Feature / Vector	Signature Antivirus	Dynamic Sandboxing	MalVision (Our System)
Analysis Latency	Milliseconds	3 - 10 Minutes	< 50 Milliseconds
Execution Risk	None (passive lookup)	High (runs hostile code)	Zero (read-only mapping)
Sandbox Evasion	Not Applicable	Highly Vulnerable (sleep tricks)	Immune (code never executes)
Polymorphic Resilience	Fails completely	Moderate	High (shared macro textures)
Explainability	None	Audit log traces	Grad-CAM Visual Heatmaps

3. Theoretical Foundations: Binary-to-Image Conversion

Any digital binary file is an ordered array of 8-bit unsigned bytes: $b_i \in [0, 255]$. In 8-bit grayscale imaging, pixel brightness is defined identically: $p_i \in [0, 255]$ (where 0 = pure black, 255 = pure white). Following the seminal methodology of Nataraj et al. (2011), the byte array is reshaped into a 2D matrix using a file-size-to-width heuristic table. For a file with N bytes and assigned width W , height is $H = \lceil N / W \rceil$, and remaining cells $(H \times W - N)$ are zero-padded (black pixels):

File Size Range	Width (W)	Architectural Rationale & Visual Properties	File Size Range	Width (W)	Architectural Rationale & Visual Properties
< 10 KB	32 px	Prevents 1-pixel line collapse.	100 - 200 KB	384 px	Accommodates multi-section droppers.
10 - 30 KB	64 px	Preserves row counts for dialers.	200 - 500 KB	512 px	Balanced aspect ratio for trojans.
30 - 60 KB	128 px	Captures header-to-code shifts.	500 KB - 1 MB	768 px	Preserves distinct code vs icon blobs.
60 - 100 KB	256 px	Standard resolution for PE tools.	> 1 MB	1024 px	High definition for complex suites.

Visual Anatomy of PE Binary Sections

- **PE Header / DOS Stub:** High-contrast geometric alternating stripes containing signature 'MZ' bytes and section offsets.
- **.text Section (Machine Opcodes):** Fine-grained, speckled 'sand/static' pattern caused by frequent x86 jumps, calls, and instructions.
- **.rdata & String Tables:** Distinct horizontal striated bands containing ASCII messages, URL paths, and registry keys.
- **.data Section:** Uniform dark blocks representing uninitialized variables, global buffers, and padding zeros.
- **Packed / Encrypted Payloads:** Chaotic, high-entropy 'TV static snow' caused by encryption randomizing byte frequencies.

4. ResNet-18 Deep Convolutional Backbone

We utilize an adapted **ResNet-18 (Residual Network)** with identity shortcut connections: $H(x) = F(x) + x$, allowing deep gradient propagation without vanishing. We adapted the first convolutional layer (conv1) by averaging pretrained ImageNet weights across the 3 RGB channels: $W_{\text{gray}} = \frac{1}{3} \sum_{c=0}^2 W_{\text{RGB}}$, enabling the model to accept 1-channel grayscale inputs while retaining all learned edge and texture filters.

Layer Block	Output Dimension	Spatial Semantics & Learned Features
Input Tensor	1 x 224 x 224	Normalized grayscale byteplot (mean=0.5, std=0.5 scaling).
Conv1 + MaxPool	64 x 112 x 112	Low-level features: Sharp section boundaries and padding runs.
Residual Layer 1	64 x 56 x 56	Opcode static: Instruction sequences and repetitive loop structures.
Residual Layer 2	128 x 28 x 28	Micro-textures: String tables, data boundaries, and import directory tables.
Residual Layer 3	256 x 14 x 14	Macro structure: Proportions of loader stub vs. encrypted payload sections.
Residual Layer 4	512 x 7 x 7	Malware family signatures: Distinct family architectural blueprint.
Adaptive AvgPool	512 x 1 x 1	Global pooling: Flattens 2D spatial features into a 512-dimensional vector.
FC Classification Head	26 Classes	Softmax probability distribution: $\sum_{j=1}^{26} P(\text{Family}_j) = 1.0$.

7. Explainable AI: Grad-CAM Decision Heatmaps

To eliminate the 'black box' problem, MalVision implements **Grad-CAM (Gradient-Weighted Class Activation Mapping)**. It backpropagates gradients from the predicted class score to layer4 feature maps, calculates channel weights α_k , and projects a thermal overlay (Red = highest attention, Blue = lowest attention) over the original binary byteplot:

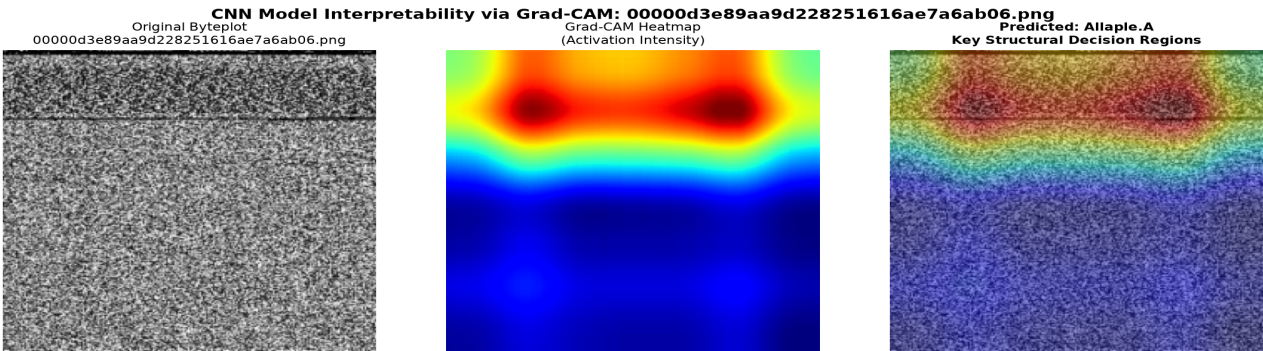


Figure C: 3-Panel Grad-CAM Output (Original Byteplot → Activation Heatmap → Decision Overlay)

8. Automated Threat Triage Levels

Threat Level	Confidence	Operational Verdict & Incident Response Action
CRITICAL	> 85%	Definite match to known malware family. Isolate host and quarantine binary immediately.
SUSPICIOUS	60% - 85%	Probable variant or modified sample. Route to secondary sandbox for behavioral audit.
INCONCLUSIVE	< 60%	Likely benign Windows executable (e.g. cmd.exe) or unindexed clean utility.

9. Project Defense & Viva Q&A; Cheat Sheet

Q: Why convert binaries to images instead of using disassembled opcodes (IDA Pro)? A: Disassembling code requires complex static parsing, takes seconds to minutes, and fails when binaries are packed or obfuscated with anti-disassembly tricks. Image conversion takes under 10 milliseconds, reads raw byte values directly, and bypasses disassembly failures entirely.
Q: What prevents malware authors from modifying their code to evade your CNN? A: While authors easily change variable names or insert junk opcodes to alter hash signatures, doing so only alters minor micro-textures. The macro-level section blueprint (e.g. tiny code stub + massive encrypted overlay) remains visually consistent across the entire family.
Q: How does the system handle clean / benign Windows files like cmd.exe? A: Through confidence thresholding and entropy analysis. When clean software is tested, its visual layout does not match any malicious family. Softmax probabilities scatter thinly across all classes, keeping the top score below 60% and triggering an INCONCLUSIVE / BENIGN triage verdict.
Q: How did you adapt a pretrained 3-channel RGB model to 1-channel Grayscale? A: We modified the first convolutional layer (conv1). The original ImageNet weights had shape [64, 3, 7, 7]. We averaged the weights across the 3 RGB channel dimensions into shape [64, 1, 7, 7], retaining all pretrained edge-detection filters while accepting single-channel grayscale input.